



Seletiva para Maratona de Programação de 2025

Instituto de Matemática e Estatística
Universidade de São Paulo

Caderno de Problemas

Apoio e patrocínio:

alura

Departamento de Ciência da Computação IME-USP
Sábado, 16 de Agosto de 2025.

Instruções

Regras

- A competição tem duração de 5 horas;
- Os times são compostos de no máximo 3 integrantes;
- Os times podem utilizar somente um computador;
- Faltando 1 hora para o término da competição, o placar será congelado, ou seja, o placar não será mais atualizado;

Entrada e Saída

- A entrada de cada problema deve ser lida da entrada padrão (teclado);
- A saída de cada problema deve ser escrita na saída padrão (tela);
- Siga o formato apresentado na descrição da saída, caso contrário não é garantido que seu código será aceito;
- Na saída, toda linha deve terminar com o caractere ‘\n’.

Índice de Problemas

- A. O Mercado Yuyuan
- B. Busca pelo Equilíbrio
- C. Ecos da Biblioteca de Jade
- D. Os Selos de Xangai
- E. Complexidade do Quicksort
- F. Inovação Chinesa
- G. Pares Incompatíveis
- H. A Sabedoria do Mestre Wei
- I. Banho
- J. O Disfarce do Mensageiro
- K. Hater de Bolos
- L. Game of Life
- M. Nômade

Problem A. O Mercado Yuyuan

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

No movimentado mercado de Yuyuan, o jovem Li quer comprar jianzhi (recortes de papel tradicionais) de um artesão. As peças estão alinhadas em uma fileira, cada uma com um símbolo único, e Li só pode comprá-las em pares idênticos seguindo estas regras:

- Se Li escolher um jianzhi na posição i , ele deve comprar imediatamente seu par em uma posição $j > i$.
- Depois de selecionar um jianzhi, ele não pode voltar para comprar um anterior.

O preço que Li paga é a soma das distâncias $j - i$ entre os pares comprados. Li quer comprar o maior número possível de pares em um único percurso. Ademais, entre todas as formas que garantam o máximo de pares, ele quer pagar o menor preço possível. Ajude Li a otimizar a sua compra.

Input

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$) — o número de símbolos nos jianzhi na fileira.

A seguinte linha contém $2N$ inteiros A_i ($1 \leq A_i \leq N$) representando os símbolos dos jianzhi da fileira, em ordem. Cada número entre 1 e N aparece exatamente duas vezes na sequência.

Output

Imprima dois inteiros separados por um espaço: o máximo número de pares que Li pode comprar e o menor preço possível para comprar essa quantidade.

Examples

standard input	standard output
3 1 2 3 2 3 1	1 2
3 1 1 2 3 2 3	2 3
6 1 2 2 3 4 4 5 3 5 1 6 6	4 5

Problem B. Busca pelo Equilíbrio

Input file: `standard input`
Output file: `standard output`
Time limit: 1.5 seconds
Memory limit: 1024 megabytes

Ying é fã de jogos e está desenvolvendo um novo RPG. Nesse jogo, cada participante começa com uma personagem que tem 0 pontos de ataque e 0 pontos de defesa. Para customizar a personagem, os jogadores podem usar *power-ups*.

Sabe-se que o jogo terá N power-ups, e cada um só pode ser usado uma vez. Cada power-up é definido por um par de inteiros (a, d) , que modifica os pontos de ataque e defesa. Se um jogador com ataque A e defesa D escolhe um power-up (a, d) os atributos finais da sua personagem serão $A + a$ de ataque e $D + d$ de defesa. Observe que os valores a e d de um power-up podem ser negativos, o que significa que os atributos finais de ataque e defesa também podem se tornar negativos.

Os desenvolvedores do jogo dizem que uma personagem é “equilibrada” se seus pontos de ataque, A , estiverem no intervalo $[A_1, A_2]$ e seus pontos de defesa, D , estiverem no intervalo $[D_1, D_2]$.

Ying quer saber de quantas maneiras um jogador pode obter uma personagem equilibrada. Uma “maneira” é definida por um **conjunto distinto de power-ups**. Dois conjuntos são distintos se um contiver um power-up que o outro não contém. A ordem em que os power-ups são aplicados não importa.

Input

A primeira linha contém um número inteiro N ($1 \leq N \leq 35$), o número de power-ups no jogo. As N linhas seguintes descrevem os power-ups. Cada uma dessas linhas contém dois números inteiros a e d ($-10^9 \leq a, d \leq 10^9$). A última linha contém quatro números inteiros A_1, D_1, A_2 e D_2 ($-10^{12} \leq A_1 \leq A_2 \leq 10^{12}$ e $-10^{12} \leq D_1 \leq D_2 \leq 10^{12}$), que definem os intervalos para uma personagem ser considerada equilibrada.

Output

Imprima uma única linha contendo um número inteiro que representa o número de conjuntos distintos de power-ups que resultam em uma personagem equilibrada.

Examples

standard input	standard output
8 1 1 1 -1 -1 1 -1 -1 1 1 1 -1 -1 1 -1 -1 -1000 -1000 1000 1000	256
8 0 0 1 1 2 1 2 0 3 0 1 1 0 2 5 5 5 3 5 3	2

Problem C. Ecos da Biblioteca de Jade

Input file: standard input
Output file: standard output
Time limit: 4 seconds
Memory limit: 512 megabytes

Após uma missão entre Xangai e Tianjin, o espião Lian retorna à sua base secreta nos becos de Xangai. Lá, recebe uma nova missão: guardar a lendária Biblioteca de Jade, oculta no vale de Longmen.

A biblioteca abriga N pergaminhos numerados de 1 a N , cada um contendo uma string composta apenas por letras minúsculas. Segundo os magos da corte, certos substratos palindrômicos ocultos nesses textos contêm instruções ancestrais e segredos mágicos. A presença desses palíndromos garante o equilíbrio da biblioteca e protegê-la significa identificar corretamente onde e quantos deles existem.

Lian deverá, portanto, responder a Q consultas. Cada consulta define um intervalo $[L, R]$ de pergaminhos e solicita o número de substrings palindrômicas distintas (de comprimento ≥ 1) que ocorrem em pelo menos um dos textos desse intervalo. Uma resposta incorreta pode ativar armadilhas deixadas pelos antigos protetores.

As consultas são feitas em tempo real e exigem resposta imediata!

Nota: Um palíndromo é uma sequência de letras que pode ser lida da mesma forma da esquerda para a direita e da direita para a esquerda. Ex.: a, aa, aba, abba. Apenas substrings palindrômicas (trechos contínuos) são consideradas.

Input

A primeira linha contém dois inteiros N e M ($1 \leq N \leq 10^5$, $1 \leq M \leq 2 \cdot 10^5$) — o número de pergaminhos e o número de consultas mágicas.

As próximas N linhas contêm os textos mágicos $s_1, s_2, \dots, s_N$, cada um formado apenas por letras minúsculas a–z. Nenhuma string é vazia, e o total de caracteres somando todas as strings é no máximo $5 \cdot 10^5$.

As M linhas seguintes contêm dois inteiros L e R ($1 \leq L \leq R \leq N$) representando a faixa de pergaminhos que os magos estão investigando.

Output

Para cada consulta, imprima uma única linha com o número de substrings palindrômicas distintas encontradas em pelo menos uma das strings entre os índices L e R .

Lembre-se: como as consultas são online, você deve fazer o flush da saída após cada linha (por exemplo, `fflush(stdout)` em C ou `cout.flush()` em C++).

Example

standard input	standard output
3 3	5
abc	5
aaa	6
aba	
1 2	
2 3	
1 3	

Problem D. Os Selos de Xangai

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 256 megabytes

Depois de sua última missão, Lian, o espião, retorna a Xangai. Em um antigo cofre de pedra, ele encontra uma sequência de números que protegem um segredo ancestral: os **Selos de Xangai**.

Para desbloquear os selos, Lian deve seguir um ritual preciso. Inicialmente escolhe um inteiro fixo M tal que $1 < M \leq 10^9$. Depois, ele deve realizar uma sequência de rodadas mágicas. Em cada rodada, ele escolhe um inteiro x com o qual desbloqueia todos os selos cujo número A_i satisfaz $A_i \bmod M = x$.

Cada rodada exige concentração e energia. Por isso, Lian quer minimizar o número de rodadas para desbloquear todos os selos. Além disso, ele quer saber quantos valores distintos de M permitem alcançar esse número mínimo de rodadas.

Input

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$), o número de selos.

A segunda linha contém N inteiros $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$), representando os números dos selos.

Output

Imprima dois inteiros: o número mínimo de rodadas necessário para desbloquear todos os selos e o número de valores distintos de M com os quais é possível alcançar esse número mínimo de rodadas.

Examples

standard input	standard output
5 13 23 33 53 103	1 3
2 2 4	1 1

Problem E. Complexidade do Quicksort

Input file: standard input
Output file: standard output
Time limit: 3 seconds
Memory limit: 256 megabytes

Na cidade de Xangai, o jovem programador Kai busca a sabedoria de seu mentor, o Mestre Wei. Um dia, Mestre Wei apresentou a Kai um desafio sobre a análise de algoritmos, uma lição fundamental para alcançar a maestria.

“Pequeno Kai,” disse o Mestre, “a força de um programador não está em escrever código, mas em compreendê-lo profundamente. Observe este pergaminho. Ele descreve o ‘Método do Particionamento Sábio’, uma forma de ordenação de sequências.”

No pergaminho, estava um algoritmo que, por coincidência, se assemelha muito a uma implementação do Quicksort em Python:

```
cmps = 0
def quicksort(A, l, r):
    if r <= l:
        return
    # O Mestre escolhe o elemento do meio como guia
    m = (l + r) / 2
    k = pivot(A, l, r, A[m])
    quicksort(A, l, k - 1)
    quicksort(A, k + 1, r)

def pivot(A, l, r, x):
    # Comparamos cada elemento com o guia x
    for y in A[l : r + 1]:
        cmps += 1
        if y < x:
            A[l] = y
            l += 1
        elif y > x:
            A[r] = y
            r -= 1
    A[l] = x
    return l
```

A função $\text{pivot}(A, l, r, x)$ rearranja os valores no segmento $A[l..r]$ da seguinte forma: os valores menores que x são movidos para $A[l..k-1]$ (na mesma ordem relativa), os valores maiores que x são movidos para $A[k+1..r]$ (na ordem relativa reversa), e o valor guia x é colocado em $A[k]$.

Mestre Wei então entregou a Kai uma sequência de números representando os níveis de energia de um dragão adormecido. “Um aprendiz apressado tentaria executar este código para obter a resposta. Mas um verdadeiro mestre pode prever o custo sem executar uma única linha.”

“Sua tarefa, Kai,” concluiu o Mestre, “é me dizer o custo exato do método para esta sequência. Calcule o valor final da variável de custo cmps que seria obtido ao final da chamada $\text{quicksort}(A, 0, n-1)$. Esta é a sua lição de hoje.”

Input

Na primeira linha, um inteiro N ($1 \leq N \leq 2 \cdot 10^5$), o tamanho do vetor. Na segunda linha, N inteiros $A[0], \dots, A[N-1]$ ($|A[i]| \leq 10^9$), os elementos do vetor. Todos os elementos de A são distintos.

Output

Um único inteiro, o número total de comparações que o método do Mestre Wei fará.

Examples

standard input	standard output
1 5	0
10 9 8 7 6 5 4 3 2 1 0	25
10 1 3 5 7 9 8 6 4 2 0	54
5 5 45 2 12 21	11

Problem F. Inovação Chinesa

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

A China foi unificada, encerrando o Período dos Reinos Combatentes e inaugurando uma era de inovações, como o teletransporte. Para conectar o império, o imperador instalou teletransportadores em diversas cidades.

O império é formado por n cidades conectadas por m ruas, de forma que é sempre possível viajar entre quaisquer duas cidades utilizando apenas essas ruas. Cada rua permite tráfego nos dois sentidos e possui um custo fixo de viagem.

Além disso, existem k tipos diferentes de teletransportadores. Duas cidades podem se conectar instantaneamente por teletransporte **somente se ambas possuírem um teletransportador do mesmo tipo**. No entanto, o custo de utilização do teletransportador depende da cidade de origem, já que cada prefeito define o valor a ser pago para sair da cidade usando o dispositivo mágico. Ou seja, mesmo para o mesmo tipo de teletransportador, o custo de uso pode variar entre cidades.

O viajante japonês Mori, ao chegar à China, não compreende sua complexa estrutura, sendo incapaz de definir os melhores caminhos para viajar. Ele deseja viajar da cidade 1 até a cidade n . Qual é o menor custo possível para realizar essa viagem, considerando tanto as ruas quanto os teletransportadores disponíveis?

Input

A primeira linha da entrada contém três inteiros: n , m e k , representando respectivamente o número de cidades, o número de ruas e a quantidade de tipos distintos de teletransportadores. Os valores obedecem às restrições $2 \leq n \leq 2 \cdot 10^5$, $n - 1 \leq m \leq \min\left(\frac{n(n-1)}{2}, 2 \cdot 10^5\right)$ e $0 \leq k \leq 2 \cdot 10^5$.

Cada uma das próximas m linhas descreve uma rua existente entre duas cidades. Cada linha contém três inteiros u , v e c , indicando que existe uma rua bidirecional conectando as cidades u e v , com custo c , onde $1 \leq u, v \leq n$ e $1 \leq c \leq 10^9$. É garantido que existe pelo menos um caminho entre quaisquer duas cidades utilizando apenas as ruas, que não há duas ruas distintas ligando o mesmo par de cidades e que nenhuma rua conecta uma cidade a ela mesma.

Em seguida, para cada uma das n cidades, há uma linha contendo um inteiro t , indicando a quantidade de teletransportadores disponíveis naquela cidade, com $0 \leq t \leq 2 \cdot 10^5$. Essa linha é seguida por t pares de inteiros (u_i, c_i) , representando que existe um teletransportador do tipo u_i disponível na cidade, e que o custo para utilizá-lo a partir dessa cidade é c_i , onde $1 \leq u_i \leq k$ e $1 \leq c_i \leq 10^9$. Um mesmo tipo de teletransportador pode **não existir em nenhuma cidade**, **existir em apenas uma** ou **existir em várias cidades**.

É garantido que o mesmo teletransportador não aparece duas vezes na mesma cidade e que a soma total dos teletransportadores disponíveis em todas as cidades não excede $\min(nk, 2 \cdot 10^5)$.

Output

Imprima um único inteiro representando a distância mínima entre as cidades 1 e n .

Examples

standard input	standard output
3 2 1 1 2 5 2 3 3 1 1 4 0 1 1 3	4
6 7 2 1 2 1 2 3 3 1 3 5 2 4 2 3 5 2 5 6 7 4 6 5 1 1 7 2 1 5 2 1 0 0 0 1 1 100	6

Problem G. Pares Incompatíveis

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

No coração da Cidade Proibida, o Imperador ordenou a realização da grande “Dança Celestial” para assegurar a harmonia do império. Uma sequência de dançarinos, representando as forças primordiais, é posicionada em uma linha. Dançarinos Yang (representados por ‘(’) e dançarinas Yin (representadas por ‘)’) formam uma sequência.

Para o grande final da dança, **cada** dançarino Yang deve ser emparelhado com uma dançarina Yin que esteja posicionada mais à frente na linha. A beleza desta dança ancestral reside em sua complexidade: os caminhos dos pares podem se cruzar, simbolizando os fios entrelaçados do destino. Por isso, existem várias coreografias possíveis, com diferentes formas de emparelhar dançarinas e dançarinos. É garantido que sempre existe uma coreografia válida.

Contudo, os astrólogos imperiais alertaram que certos pares são “incompatíveis”. Um par incompatível é um dançarino Yang e uma dançarina Yin específicos que, se forem emparelhados, tornam impossível que todos os outros dançarinos encontrem um par, quebrando a harmonia cósmica do ritual.

Por exemplo, na formação `(( ))`, qualquer par que se forme (como o 1º com o 3º, ou o 1º com o 4º) sempre permite que os dançarinos restantes formem um par válido. Portanto, para `(( ))`, não há pares incompatíveis. Já na formação `()()`, se tentarmos parear o 1º dançarino (Yang) com a 4ª dançarina (Yin), os dançarinos restantes são a 2ª (Yin) e o 3º (Yang). Como a dançarina Yin restante vem antes do dançarino Yang, eles não podem formar um par. Logo, o par (1, 4) é incompatível.

Sua tarefa, como Mestre Coreógrafo do Império, é analisar a formação inicial dos dançarinos e determinar o número total de pares incompatíveis, para que possam ser evitados na cerimônia.

Input

A primeira linha contém um único inteiro N ($1 \leq N \leq 10^6$), o número de dançarinos na sequência.

A segunda linha contém uma sequência de N caracteres, representando os dançarinos. É garantido que existe uma coreografia válida.

Output

Imprima um único inteiro: o número de pares incompatíveis na sequência dada.

Examples

standard input	standard output
4 (())	0
4 ()()	1
8 ((())())	3

Problem H. A Sabedoria do Mestre Wei

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 256 megabytes

Na antiga e majestosa cidade de Xangai, futuro palco da final mundial do ICPC, um jovem programador chamado Kai sonha em alcançar a maestria de seu mentor, o lendário Mestre Wei.

Certo dia, enquanto depuravam um código complexo no Templo do Ganso Selvagem, Kai perguntou: “Mestre, sua experiência é vasta como o Rio Amarelo. Quando a minha jornada em programação será exatamente metade da sua?”

Mestre Wei, com um sorriso enigmático, respondeu: “Pequeno gafanhoto, a experiência é medida pelos anos de prática. Eu comecei a programar no ano W , e você começou no ano K . Haverá um ano exato em que meus anos de prática serão o dobro dos seus. Descubra esse ano e você encontrará a clareza que busca.”

Ajude Kai a resolver o enigma de seu mestre.

Input

A entrada consiste em uma única linha com dois inteiros W e K ($0 \leq W < K \leq 10^9$), representando o ano em que o Mestre Wei começou a programar e o ano em que Kai começou, respectivamente.

Output

Imprima um único número: o ano em que a experiência do mestre será exatamente o dobro da experiência de Kai.

Examples

standard input	standard output
1975 1995	2015
1989 2007	2025
0 1000000000	2000000000

Problem I. Banho

Input file: `standard input`
Output file: `standard output`
Time limit: 5 seconds
Memory limit: 256 megabytes

Nos tempos turbulentos dos Reinos Combatentes, o renomado guerreiro Na Tan e seus companheiros viajaram para participar de um torneio de artes marciais. Todos estão hospedados no mesmo aposento e, antes da competição, Na Tan decide ir aos banhos para clarear a mente. No entanto, ao terminar, percebe que cometeu um erro: esqueceu de pegar a toalha!

Felizmente, o aposento está vazio no momento, mas seus companheiros podem retornar a qualquer instante. Na Tan, determinado a não ser visto em uma situação constrangedora, precisa sair da sala de banho, pegar a toalha e voltar **o mais rápido possível**, sem ser surpreendido.

O aposento pode ser representado por um **polígono simples** (ou seja, um polígono cujos lados não se cruzam), e o trajeto de Na Tan deve ocorrer **inteiramente dentro desse polígono**. A porta da sala de banho está localizada em um dos **vértices do polígono**, enquanto a toalha está em algum ponto **dentro do polígono**, mas não necessariamente em um vértice. Na Tan se movimenta a uma velocidade de 1 unidade por segundo.

Dado isso, qual é o **menor tempo necessário** para Na Tan buscar a toalha e retornar à sala de banho?

Input

A primeira linha consiste de um inteiro n ($3 \leq n \leq 500$) indicando o número de pontos no polígono.

As próximas n linhas contêm dois inteiros x e y ($-10^6 \leq x, y \leq 10^6$) que representam os vértices do polígono, é garantido que o polígono é simples, os vértices são dados em sentido anti-horário do polígono e que **a porta da sala de banho está localizado no primeiro vértice do polígono**.

A última linha contém dois inteiros x e y ($-10^6 \leq x, y \leq 10^6$) que representam o ponto da toalha.

É garantido que quaisquer três pontos, **incluindo a toalha**, não são colineares.

Output

Imprima o menor tempo em segundos para Na Tan ir e voltar partindo do primeiro ponto e indo até a toalha. A resposta será considerada correta se a diferença absoluta ou relativa for menor que 10^{-4} .

Examples

standard input	standard output
4 0 0 10 0 10 10 0 10 8 9	24.083189158
6 0 20 0 0 40 0 40 50 20 40 20 20 30 32	71.240998704
8 0 80 0 0 50 0 50 60 30 60 30 20 20 20 20 80 40 33	179.293545340

Problem J. O Disfarce do Mensageiro

Input file: **standard input**
Output file: **standard output**
Time limit: 3 seconds
Memory limit: 256 megabytes

Na era dos Reinos Combatentes da antiga China, a espionagem era a chave para a sobrevivência. Lian, um agente secreto, recebeu uma missão crítica: entregar uma mensagem de sua base em **Xangai** (representada no mapa pelo vértice S) para um general aliado na cidade murada de **Tianjin** (vértice T).

O mestre espião de Lian o alertou: “Nossos inimigos são astutos. Eles conhecem todas as estradas e esperam que um mensageiro urgente use o caminho mais rápido possível. Se você chegar em tempo recorde de Xangai a Tianjin, sua identidade será revelada.”

A ordem era clara: calcule o comprimento D do caminho mais curto, mas não o utilize. Em vez disso, para se passar por um mercador comum que talvez tenha pego um desvio, você deve seguir um caminho de comprimento exatamente $D + 1$.

Para ter flexibilidade e rotas de fuga, Lian precisa saber quantas opções de trajeto ele tem que satisfazem essa condição. Sua tarefa é ajudá-lo a contar o número de caminhos distintos de Xangai a Tianjin com comprimento precisamente um a mais que o mínimo.

Input

A primeira linha contém quatro inteiros N, M, S , e T ($1 \leq N, M \leq 10^6, 1 \leq S \neq T \leq N$), representando o número de cidades e estradas na China, o vértice de partida (S , Xangai) e o vértice de destino (T , Tianjin). As M linhas seguintes contêm, cada uma, dois inteiros u_i e v_i ($1 \leq u_i \neq v_i \leq N$), indicando que existe uma estrada (de mão dupla) entre as cidades u_i e v_i .

Output

Imprima um único inteiro: o número de caminhos distintos de Xangai para Tianjin com comprimento um a mais que o mínimo. Como este número pode ser muito grande, imprima o resultado módulo $10^9 + 7$.

Examples

standard input	standard output
5 6 1 3 1 2 2 3 1 4 4 2 2 5 5 3	2
3 5 1 2 1 2 1 3 1 3 2 3 2 3	4
2 1 1 2 1 2	0

Note

É garantido que existe um caminho de Xangai para Tianjin. Podem existir múltiplas estradas entre o mesmo par de cidades.

Problem K. Hater de Bolos

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Agente Wang é uma pessoa bastante reservada e prefere não compartilhar a data do seu aniversário. No entanto, a tríade do MaratonUSP acabou descobrindo quando ele faz aniversário!

Para comemorar, a tríade decidiu fazer um bolo do zero. A responsabilidade pela receita ficou com Antonio, o cozinheiro da tríade — mas há um problema: Antonio **não** gosta de bolos.

Depois de investigar a fundo, Antonio descobriu quais ingredientes do bolo ele não gosta e resolveu simplesmente não usá-los na preparação, mesmo que isso comprometa o resultado. A receita do bolo é dividida em várias etapas, e cada etapa utiliza uma lista de ingredientes. O bolo ficará **ruim** se, em qualquer etapa, **mais** de um terço dos ingredientes **da etapa** estiver ausente.

Qual a primeira etapa que o bolo ficará **ruim**, caso exista?

Input

Na primeira linha, um inteiro N ($1 \leq N \leq 10^3$), o número de etapas.

Na segunda linha, dois inteiros A e B ($1 \leq B \leq A \leq 3 \cdot 10^5$), o número total de ingredientes e o número de ingredientes que Antonio **não** gosta.

Na terceira linha, B inteiros distintos entre 1 e A , indicando os ingredientes que Antonio **não** gosta.

Cada uma das próximas N linhas representa uma etapa, contém um inteiro M ($1 \leq M \leq A$), o número de ingredientes dessa etapa, seguida de M inteiros distintos entre 1 e A representando os ingredientes da etapa.

A soma da quantidade de ingredientes em todas as etapas é no máximo $3 \cdot 10^5$.

Output

Imprima um único inteiro, o índice da primeira etapa que o bolo ficará **ruim**, ou -1 se nenhuma etapa fará o bolo ficar ruim.

Example

standard input	standard output
3 6 3 1 3 5 4 1 2 4 6 3 2 3 4 6 6 5 4 3 2 1	3

Note

No primeiro caso de teste, há 3 etapas e 6 ingredientes, em que Antonio não gosta dos ingredientes 1, 3 e 5.

Na primeira etapa, dos 4 ingredientes, Antonio não gosta do ingrediente 1. Logo, o bolo não ficará ruim.

Na segunda etapa, dos 3 ingredientes, Antonio não gosta do ingrediente 3. Antonio retira exatamente um terço dos ingredientes; logo, o bolo não ficará ruim.

Na terceira e última etapa, dos 6 ingredientes, Antonio não gosta de metade deles. Logo, o bolo ficará ruim.

Problem L. Game of Life

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Dado um tabuleiro de tamanho $N \times N$, em que cada célula pode estar viva, morta ou ser inabitável, simule a evolução do tabuleiro por K iterações com base nas seguintes regras:

- Uma célula viva ('1') morre se tiver um número ímpar de vizinhos vivos.
- Uma célula morta ('0') torna-se viva se tiver um número ímpar de vizinhos vivos.
- Uma célula inabitável ('#') nunca muda de estado e nunca é considerada viva.

Cada célula possui até 8 vizinhos (horizontal, vertical e diagonais).

Seu objetivo é determinar o estado final do tabuleiro após K iterações.

Input

A primeira linha contém dois inteiros N e K ($2 \leq N \leq 8$, $1 \leq K \leq 10^9$) — o tamanho do tabuleiro e o número de iterações.

As próximas N linhas contêm N caracteres cada, descrevendo o estado inicial do tabuleiro.

Seja $S_{i,j}$ o caractere na i -ésima linha e j -ésima coluna:

- $S_{i,j} = '1'$ se a célula está viva,
- $S_{i,j} = '0'$ se a célula está morta,
- $S_{i,j} = '#'$ se a célula é inabitável.

Output

Imprima N linhas, cada uma com N caracteres, representando o estado final do tabuleiro após K iterações.

Examples

standard input	standard output
4 1 #101 0000 1100 1100	#101 1111 0000 0000
2 1000000000 01 ##	00 ##

Problem M. Nômade

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Devido aos preços elevados de aluguel em Xangai, Kai, do clã Yafa, ficou sem moradia. Felizmente, ele já encontrou uma nova! No entanto, precisará esperar **exatamente** D dias até poder se mudar.

Enquanto isso, ele avaliou suas opções e encontrou N lugares, representados por índices de 1 a N , onde pode passar a noite. Cada lugar, no entanto, tem um limite de tolerância: após um certo número de dias consecutivos, Kai será expulso.

Para cada um dos N lugares, foi determinado um inteiro d_i , representando o número máximo de dias consecutivos que ele pode dormir no i -ésimo local antes de ser expulso. É possível que $d_i = 0$; nesse caso, Kai julgou mal o lugar e será expulso antes mesmo de conseguir entrar.

Seu objetivo é montar um plano de estadia para que Kai consiga passar os D dias alternando entre esses lugares, **sem ultrapassar o limite consecutivo de nenhum deles**. Kai pode voltar a um lugar, desde que respeite o limite de permanência consecutiva.

Escreva um programa que determine se é possível montar esse plano. Se sim, produza uma sequência de lugares (índices de 1 a N), indicando onde Kai deve dormir em cada um dos D dias. Caso contrário, imprima -1 .

Input

A primeira linha contém dois inteiros N e D ($1 \leq N, D \leq 10^5$), representando o número de lugares e os dias que Kai precisa esperar para se mudar, respectivamente.

A segunda linha contém N inteiros $d_1, d_2, \dots, d_N$ ($0 \leq d_i \leq 10^5$), onde d_i é o limite de tolerância do i -ésimo.

Output

Imprima D inteiros, a sequência de lugares representando o plano de estadia. Se não for possível fazer um plano, imprima -1 .

Examples

standard input	standard output
3 5 0 1 3	2 3 3 3 2
10 1 0 0 0 0 0 0 0 0 0 0	-1